

ARC106C Solutions 题解

题意

有以下两个算法：

算法 A：

输入 n 个区间 $[L_i, R_i]$ ，按右端点从小到大排序，然后从前往后对于每一个区间，如果这个区间不与前面任何一个选择的区间相交则选择这个区间，否则不选择，最后输出选择的区间数。

算法 B：

输入 n 个区间 $[L_i, R_i]$ ，按左端点从小到大排序，然后从前往后对于每一个区间，如果这个区间不与前面任何一个选择的区间相交则选择这个区间，否则不选择，最后输出选择的区间数。

要求构造 n 个区间 $[L_i, R_i]$ ，使得算法 A 得出的结果减去算法 B 得出的结果刚好为 m 。数据范围 $n \leq 2 \times 10^5, m \in [-n, n]$ 。

思路 1

首先，算法 A 就是计算所有区间中最多能选出多少个两两不相交区间的算法，所以算法 B 得到的答案不可能比算法 A 更大，所以如果 m 小于 0，则一定无解。

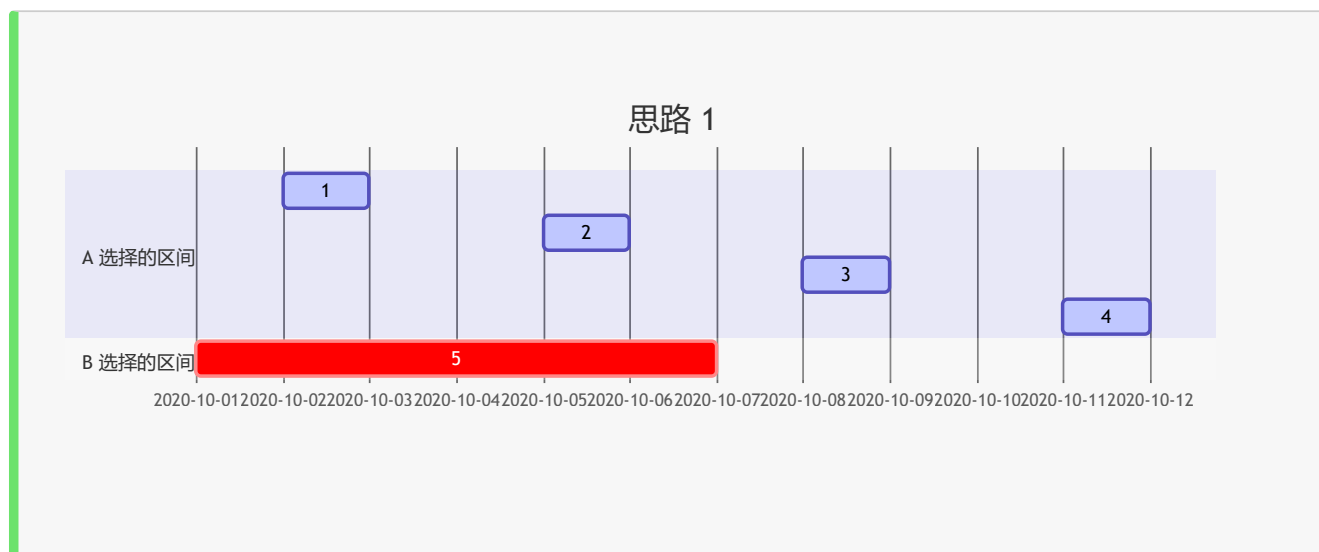
下面还有几种特殊情况，一种是 $m = n$ 的情况，即在算法 A 得出结果 n 的时候算法 B 得到的结果是 0 的情况，但是很显然，算法 B 得到的结果不可能是 0，最坏情况也只能是 1，所以这种情况也无解。

还有一种 $m = n - 1$ 的情况，这种情况下相当于如果 A 得到的结果是 n ，B 得到的结果必须是 1。但是，A 得到 n 的结果只有一种情况：所有区间互不相交的情况，而这种情况下，B 也只有可能得到 n ，不可能得到 1。

但是上面这种情况只对 $n > 1$ 有效，当 $n = 1, m = 0$ 时，任意一个区间都可以满足要求（因为 m 为 0，所以两者的结果都是 1 是符合要求的），所以这种情况还需要再特判，我一开始就是因为没有特判这种情况 WA 了一次。

最后，如果几种特殊情况都不满足，则说明有解，按照如下方式构造这个解：首先让算法 A 计算出 $n - 1$ 的结果，即构造 $n - 1$ 个互不相交的区间（第 i 个区间可以是 $[3i, 3i + 1]$ ，因为要求任意两个区间端点都不重合），然后为了让 B 计算的结果比 A 少 m ，则需要一个大

区间覆盖住前 $m + 1$ 个小区间（这样 A 算法会选择 $m + 1$ 个小区间，但是 B 算法只会选择这个大区间），这个区间可以是 $[2, 3(m + 1) + 2]$ 。最终输出这些区间即可。



代码 1 来源：我

```
n, m = map(int, input().split())
if n == 1 and m == 0:
    print(1, 2)
    exit(0)
if m < 0 or m >= n - 1:
    print("-1")
    exit(0)

for i in range(1, n): print(3 * i, 3 * i + 1)
print(2, 3 * (m + 1) + 2)
```

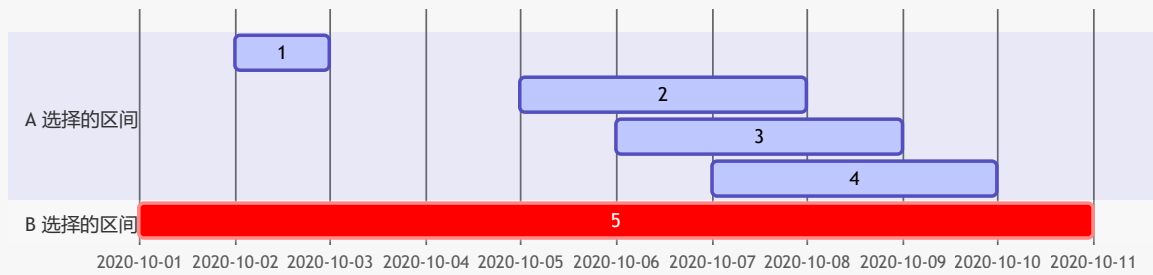
思路 2

考虑换一种构造的方式，这次不是先让 A 计算出 $n - 1$ ，而是让 B 计算出 1。所以，先用一个大区间把所有的范围全部覆盖（这样 B 只会选择这个大区间）。

接着在大区间中放上 m 个互不相交的小区间，这样 A 就会先把这 m 个小区间选上。

下面还需要让 A 再选刚好一个区间，但是还需要再构造 $n - m - 1$ 个区间，所以可以让这些区间两两相交。具体方式为：从某个位置 x 开始，连续选择 $n - m - 1$ 个位置作为区间左端点，再往后连续选择 $n - m - 1$ 个位置作为区间右端点，然后将最左边的左端点与最左边的右端点匹配成一个区间，这样 A 只会选择其中一个区间，就达到了目的。

思路 2



代码 2 来源: Um_nik

```
#include <iostream>
#include <cstdio>
#include <cstdlib>
#include <algorithm>
#include <cmath>
#include <vector>
#include <set>
#include <map>
#include <unordered_set>
#include <unordered_map>
#include <queue>
#include <ctime>
#include <cassert>
#include <complex>
#include <string>
#include <cstring>
#include <chrono>
#include <random>
#include <bitset>
using namespace std;

#ifdef LOCAL
    #define eprintf(...) fprintf(stderr, __VA_ARGS__);fflush(stderr);
#else
    #define eprintf(...) 42
#endif

using ll = long long;
using ld = long double;
using uint = unsigned int;
```

```

using ull = unsigned long long;
template<typename T>
using pair2 = pair<T, T>;
using pii = pair<int, int>;
using pli = pair<ll, int>;
using pll = pair<ll, ll>;
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll myRand(ll B) {
    return (ull)rng() % B;
}

#define pb push_back
#define mp make_pair
#define all(x) (x).begin(),(x).end()
#define fi first
#define se second

clock_t startTime;
double getCurrentTime() {
    return (double)(clock() - startTime) / CLOCKS_PER_SEC;
}

int main()
{
    startTime = clock();
    // freopen("input.txt", "r", stdin);
    // freopen("output.txt", "w", stdout);

    int n, m;
    scanf("%d%d", &n, &m);
    if (n == 1) {
        if (m != 0) {
            printf("-1\n");
        } else {
            printf("1 2\n");
        }
        return 0;
    }
    if (m < 0 || m > n - 2) {
        printf("-1\n");
        return 0;
    }
    printf("1 1000000\n");
    int x = 2;

```

```
int k = n - m - 2;
for (int i = 0; i < k + 1; i++)
    printf("%d %d\n", x + i, x + k + 1 + i);
x += 2 * (k + 1);
for (int i = 0; i < m; i++) {
    printf("%d %d\n", x, x + 1);
    x += 2;
}

return 0;
}
```